# Introduction

# Introduction I

Disclosure control—the process of sanitizing classified or sensitive information before public release—requires multiple layers of validation to ensure that no source identities, operational details, time-place selectors, or controlled dissemination markers leak into the public record. This exemplar implements a reproducible pipeline that combines text-level disclosure control with visual redaction proofing, steganographic provenance embedding, and hardware-backed TPM sealing.

The pipeline operates on invented fixture data: synthetic segments with classification levels ranging from UNCLASSIFIED through TOP_SECRET//SCI, synthetic redaction decisions with bounded reasons, and synthetic reviewer records. No real source material is used. The exemplar demonstrates the full release-review workflow: classification-ceiling enforcement, redaction-span validation, source-control coverage, mosaic-risk scoring, residual-marker detection, review-gate evaluation, source-safe ledger generation, and TPM-sealed sidecar production.

## Scope and Safety Boundaries I

This exemplar is limited to lawful redaction, declassification support, public-records release review, taxonomy normalization, source-safe ledgers, reviewer approval gates, sanitized packet export, and source-protection auditing. It does not provide targeting, collection, evasion, or surveillance operational guidance. All committed examples are invented fixtures.

# Contributions I

1. A classification taxonomy with SCI alias support and configurable public ceilings.
2. A release-audit engine that validates redaction spans, checks orphan decisions, enforces source-control coverage, and scores mosaic risk.
3. A source-safe redaction ledger that records SHA-256 hashes of redacted spans without exposing source text.
4. A visual proof matrix that enumerates four redaction styles across four PDF backgrounds, producing sixteen variant PDFs with identical source-safe decisions.
5. A steganography layer that post-processes each base PDF with nine security methods, producing provenance-enhanced companion PDFs with hash manifests.
6. Optional Kmyth TPM sealing that wraps each hash manifest and steganography PDF in `.ski` sidecars sealed against the TPM2-TSS storage hierarchy.
7. A comprehensive in-memory release packet plus deterministic public projections: a text-free audit and a hashed source-safe ledger.

# Architecture

## Architecture I

The pipeline architecture consists of two layers. Layer one performs text-level disclosure control: each segment is audited against a public classification ceiling, redaction spans are validated for non-overlap, orphan decisions are flagged, and source-control coverage is enforced. Layer two applies visual redaction treatments, steganographic provenance overlays, and optional Kmyth TPM sidecar sealing.

# Architecture I

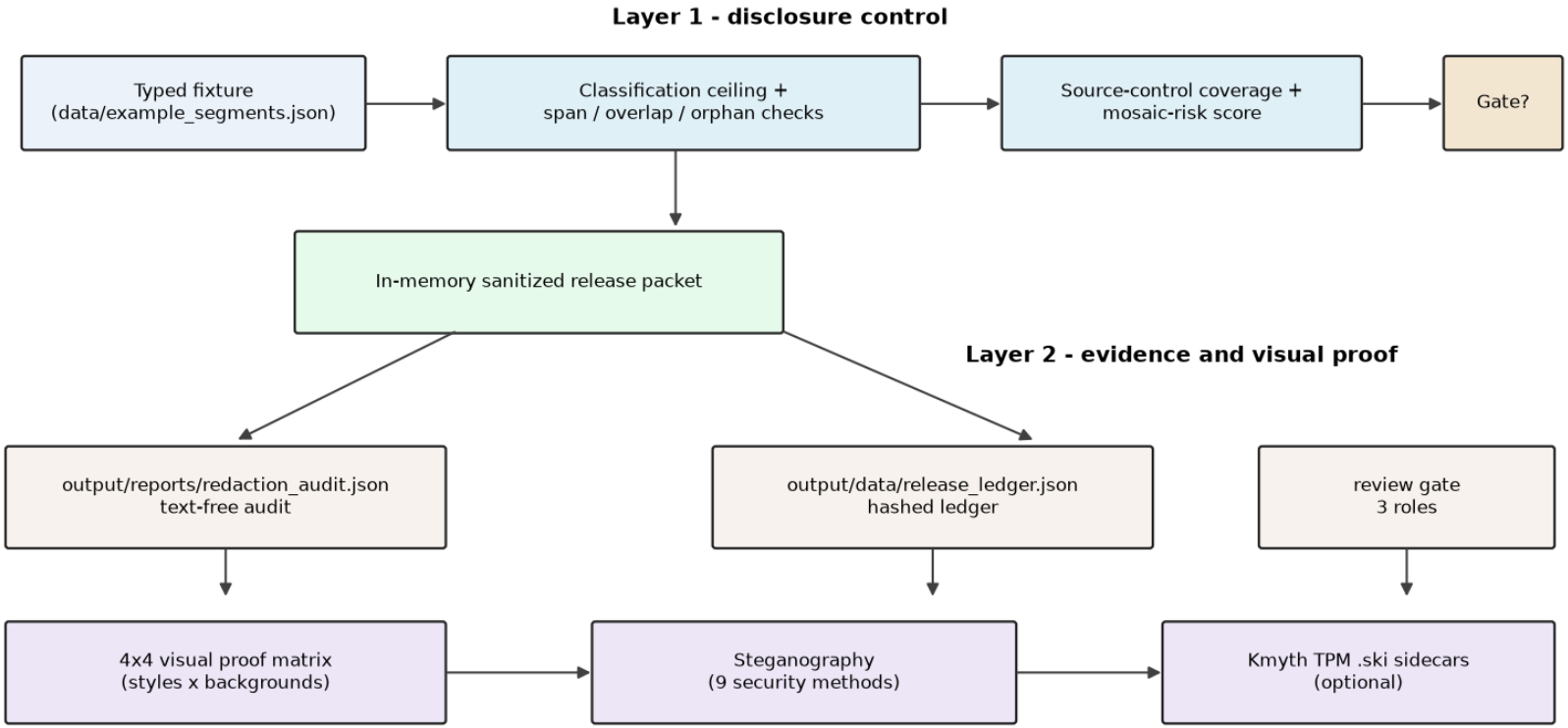


Figure 1: Release-review pipeline: the disclosure-control layer audits the invented fixture against the public ceiling and builds an in-memory sanitized packet, the evidence layer projects text-free audit and hashed-ledger JSON, and the visual layer renders the 4x4 proof matrix with optional steganography and Kmyth TPM sidecars.

## Architecture I

fig. 1 traces the full pipeline: typed fixture load, ceiling and span validation, source-control coverage and mosaic checks, the in-memory sanitized packet, the two text-free public projections, and the visual proof chain.

## Layer One: Disclosure Control I

The disclosure-control engine operates on `RedactionSegment` objects, each carrying an identifier, classification level, text content, and optional source controls. Redaction decisions are `RedactionDecision` records with bounded reasons: `source_identity`, `operational_detail`, `time_place_selector`, `legal_privilege`, and `privacy`. The audit engine validates that:

# Layer One: Disclosure Control I

- Redaction spans are non-overlapping and within text bounds.
- Each segment above the public ceiling has at least one redaction decision.
- Each source control has a corresponding `source_identity` redaction.
- No orphan decisions reference missing segments.
- Residual markers (email, IP, coordinates, NOFORN, HUMINT, SIGINT) are absent from sanitized text.
- The mosaic risk score—residual markers normalized by segment and pattern count—does not exceed the policy threshold.

## Layer Two: Visual Proof and Steganography I

Visual proofing is parameterized separately from the release-audit text path. Four redaction styles—blackout (solid black fill), whiteout (white fill with gray text), grayout (mid-gray fill), and blur (offset-rendered token)—are rendered across four PDF backgrounds—white, gray, black, and blur (subdued text with blur effect). The $4 \times 4$ matrix yields sixteen variant PDFs, each receiving the same source-safe redaction decisions.

## Layer Two: Visual Proof and Steganography I

The steganography layer post-processes each base PDF with nine security methods:

# Layer Two: Visual Proof and Steganography I

| Method | Purpose |
|---|---|
| SHA-256/SHA-512 hash manifest | Cryptographic integrity verification |
| Diagonal watermark overlay | Visible provenance stamp |
| Footer provenance overlay | Page-level release metadata |
| First-page invisible text | Hidden identification marker |
| QR payload barcode | Machine-readable provenance |
| Code128 page barcode | Per-page tracking barcode |
| PDF Info metadata | Document-level metadata injection |
| XMP metadata | Standards-compliant metadata embedding |

# Layer Two: Visual Proof and Steganography I

| Method | Purpose |
| --- | --- |
| Embedded stego manifest attachment | Self-contained provenance archive |

## Layer Three: Kmyth TPM Sealing I

Kmyth TPM sealing wraps each hash manifest and steganography PDF in a `.ski` sidecar sealed against the TPM2-TSS storage hierarchy. The sealed objects are bound to PCR selections and policy or-values, ensuring that unsealing requires the same platform configuration.

On macOS, which lacks a hardware TPM, a software TPM emulator (swtpm) and an mssim-to-swtpm protocol proxy bridge the TPM2-TSS mssim TCTI to swtpm's native socket protocol. The proxy:

# Layer Three: Kmyth TPM Sealing I

- **Control channel**: intercepts MS simulator platform commands (POWER_ON=1, NV_ON=11, TPM_SESSION_END=20) and returns success, since swtpm's `--flags startup-clear` handles TPM initialization.
- **Data channel**: strips the 9-byte mssim wire header (4B cmd_type + 1B locality + 4B tpm_size), forwards raw TPM commands to swtpm, and wraps responses back in mssim format.

## Layer Three: Kmyth TPM Sealing I

The kmyth-seal binary was patched to call `Tss2_Sys_FlushContext` for the storage key handle before freeing TPM2 resources. Without this patch, consecutive kmyth-seal invocations exhaust the swtpm transient object slots (typically three), causing "out of memory for object contexts" errors on the second seal.